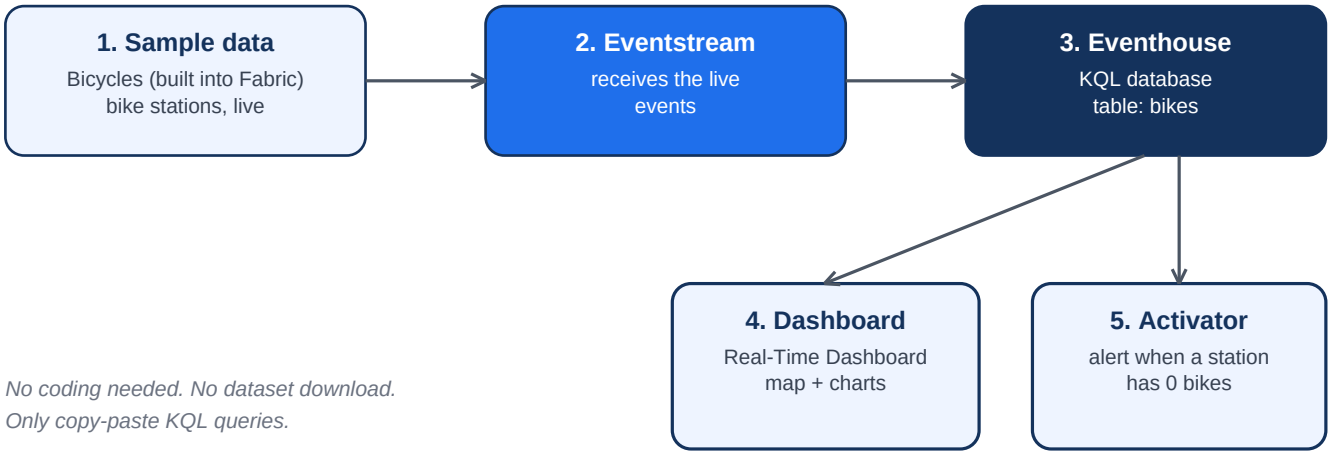
Live Bike-Station Monitoring in Microsoft Fabric

An EASY live-streaming project: no coding, no dataset download, about 2-3 hours

What you build	A live stream of bike-station data into Fabric, a live dashboard with a map, and an automatic alert when a station runs out of bikes.
Why it is easy	Fabric gives you the live data for free (built-in sample). You only click in the portal and paste a few KQL queries.
Skills you prove	Eventstream, Eventhouse, KQL, Real-Time Dashboard, Activator, Git.
Real-world story	Bike, scooter, taxi and delivery companies monitor stations and vehicles live to rebalance supply. The same pattern is used for IoT and fleet tracking.
Cost	Free (Fabric trial).
Final output	GitHub repo with queries, screenshots and README.

TIP: This is the simple version. After you finish it, you can go to the bigger fraud project and it will feel easy, because the steps are the same: Eventstream, Eventhouse, KQL, Dashboard, Activator.

Part 1: Big Picture



No coding needed. No dataset download.
Only copy-paste KQL queries.

What happens, in simple words

1. Fabric creates fake-but-realistic **bike station data** every second (how many bikes and empty docks each station has).
2. **Eventstream** receives this live data.
3. It stores the data in an **Eventhouse** (a database made for live data).
4. You write **KQL** queries to answer questions: which stations are empty? which area has most bikes?
5. A **Real-Time Dashboard** shows the answers live, with a map.
6. **Activator** sends you an email when a station has no bikes.

Words to know

Word	Meaning
Eventstream	Fabric tool that receives live events and sends them to destinations.
Eventhouse	Database for fast queries on live, time-based data. It contains KQL databases.
KQL	Query language used in Eventhouse. Similar idea to SQL: Table | where | summarize.
Activator	Watches data and triggers email/Teams when a condition is true.

Part 2: What You Need

- **Fabric free trial**: https://learn.microsoft.com/fabric/fundamentals/fabric-trial
- **Fabric portal**: https://app.fabric.microsoft.com
- **GitHub account** (free) and **Git** installed.
- A browser. That is all. No Python needed for this project.

Useful links

Topic	Link
Real-Time Intelligence tutorial (same sample data)	https://learn.microsoft.com/fabric/real-time-intelligence/tutorial-introduction
Eventstreams overview	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview
Eventhouse	https://learn.microsoft.com/fabric/real-time-intelligence/eventhouse
KQL quick reference	https://learn.microsoft.com/kusto/query/kql-quick-reference
Real-Time Dashboards	https://learn.microsoft.com/fabric/real-time-intelligence/dashboard-real-time-create
Activator	https://learn.microsoft.com/fabric/real-time-intelligence/data-activator/activator-introduction

CAREFUL: Microsoft moves pages sometimes. If a link fails, search the page title. Fabric screens also change a little, so a button name may differ; the idea is the same.

Step 1: Start Trial and Create Workspace

1. Go to https://app.fabric.microsoft.com and sign in.
2. Click your profile picture (top right), then **Start trial**. Accept defaults.
3. Left menu: **Workspaces**, then **New workspace**.
4. Name: `bike-rti`. License mode: **Trial**. Click Apply.

CHECK: you are inside the empty workspace bike-rti.

Step 2: Create the Eventhouse

1. In the workspace click **New item**, then choose **Eventhouse**.
2. Name: `bike_eh`. Click Create.
3. Wait a minute. A KQL database named bike_eh is created automatically inside it.

CHECK: you can see the Eventhouse page with the database bike_eh.

Step 3: Create the Eventstream with Sample Data

1. In the workspace click **New item**, then **Eventstream**. Name: `bike_stream`.
2. On the canvas choose **Use sample data** (or Add source, then Sample data).
3. Source name: `bike_source`. Sample data: choose **Bicycles**. Click Add.
4. Wait until the source shows as connected. Click the middle stream node and open the **Data preview** tab.
5. You should see rows with columns like BikepointID, Street, Neighbourhood, Latitude, Longitude, No_Bikes, No_Empty_Docks.

TIP: Write down the exact column names you see in Data preview. All queries in this guide use the names above. If your names are slightly different, change them in the queries.

CHECK: Data preview shows rows that keep changing.

Step 4: Send the Data to the Eventhouse

1. In the Eventstream click **Edit** (if not already editing), then **Add destination**, then **Eventhouse**.
2. Ingestion mode: **Direct ingestion**. Destination name: `to_eventhouse`.
3. Select workspace bike-rti, Eventhouse **bike_eh**, KQL database **bike_eh**. Click Save.
4. Click **Publish**. Then in the destination pane choose **Configure** (or Open event processor / setup wizard).
5. Choose **New table**, name it `bikes`. Data format: **JSON**. Check the columns look right and click Finish.
6. Wait 1-2 minutes for ingestion to start.

Check the data arrived

Open the KQL database (Eventhouse, then bike_eh) and click **Explore your data**. Run:

```
bikes
| take 10

bikes
| count
```

Run the count again after a minute. If the number grows, live data is flowing.

WHY THIS MATTERS: The bikes table might not show a time column. We use `ingestion_time()` in the queries, which is the time Fabric received each row. It works on every table, so you do not depend on any column name.

COMMON ERROR: If bikes has 0 rows after 3 minutes: go back to the Eventstream, make sure it is Published, and that the destination status is Active. Re-open the destination and check the table name is bikes.

CHECK: `bikes | count` keeps increasing.

Step 5: Write the KQL Queries

Create a **KQL Queryset**: in the workspace choose New item, **KQL Queryset**, name it `bike_queries`, and connect it to the bike_eh database. Run each query separately (select it, press Shift+Enter). Save all of them later in `kql/queries.kql` in your repo.

Query 1: Latest status of every station

Stations send many updates. This keeps only the newest row per station.

```
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
```

Query 2: Stations with no bikes (the alert case)

```
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
| where No_Bikes == 0
| project BikepointID, Street, Neighbourhood, No_Bikes, No_Empty_Docks
```

Query 3: Stations that are completely full (no free docks)

```
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
| where No_Empty_Docks == 0
| project BikepointID, Street, Neighbourhood, No_Bikes, No_Empty_Docks
```

Query 4: Bikes available by neighbourhood

```
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
| summarize totalBikes = sum(No_Bikes), emptyDocks = sum(No_Empty_Docks),
            stations = count() by Neighbourhood
| order by totalBikes desc
```

Query 5: Total bikes over time (trend line)

```
bikes
| where ingestion_time() > ago(30m)
| summarize avgBikes = avg(No_Bikes) by t = bin(ingestion_time(), 1m)
| render timechart
```

Query 6: Map data

```
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
| project Latitude, Longitude, Street, No_Bikes, No_Empty_Docks
```

Query 7: Small KPI numbers

```
// Stations with no bikes right now
bikes
| extend ingest = ingestion_time()
| summarize arg_max(ingest, *) by BikepointID
| where No_Bikes == 0
| count

// Events received in last 5 minutes
bikes
| where ingestion_time() > ago(5m)
| count
```

WHY THIS MATTERS: KQL cheat sheet: **where** = filter, **summarize ... by** = GROUP BY, **project** = SELECT columns, **extend** = add a column, **order by** = sort, **take 10** = TOP 10, **bin(x, 1m)** = group time into 1-minute buckets, **arg_max(col, *)** = latest row per group.

COMMON ERROR: If you get an error like 'column not found', run `bikes | getschema` to see real column names and fix the query.

CHECK: Queries 1, 2, 4 and 6 return rows.

Step 6: Build the Real-Time Dashboard

1. Workspace, **New item**, **Real-Time Dashboard**. Name: `bike_dashboard`.
2. Click **New data source** and select the KQL database **bike_eh**.
3. Click **Add tile**. Paste a query, click **Run**, then **Add visual**, choose the visual type, and Apply changes.
4. Add the tiles listed in the table below.
5. Turn on **Auto refresh** at the top (e.g. 30 seconds). Click Save.

Tile name	Query	Visual
Stations with no bikes	Query 7 (first one)	Stat / card
Events last 5 min	Query 7 (second one)	Stat / card
Station map	Query 6	Map (Latitude, Longitude, size by No_Bikes)
Bikes by neighbourhood	Query 4	Bar chart
Bikes trend	Query 5	Time chart
Empty stations list	Query 2	Table
Full stations list	Query 3	Table

TIP: Take screenshots now while data is moving: full dashboard, map, and each chart. Recruiters see screenshots first. Save them in the dashboards/ folder.

CHECK: numbers change every time the dashboard refreshes.

Step 7: Activator Alert

Goal: get an email when stations have no bikes.

1. Open the dashboard. On the tile **Stations with no bikes** click the three dots (...) and choose **Set alert**.
2. Condition: when the value is **greater than 0** (if it is always more than 0 in your data, use a bigger number such as 3 so you can see it trigger).
3. Action: **Send me an email**. Subject: *Bike stations are empty*.
4. Save location: choose your workspace and a new Activator item named `bike_alerts`. Create.
5. Make sure the rule is **started** (open the Activator item and check).

TIP: The first email can take a few minutes. Check spam too. Take a screenshot of the email for your README.

CAREFUL: Do not create many alerts. One or two clean alerts are enough for the demo and keep trial capacity free.

CHECK: you received one alert email.

Step 8: Put the Project on GitHub

Repo structure

```
fabric-bike-realtime/
|-- README.md
|-- docs/architecture.png
|-- kql/queries.kql
|-- dashboards/
    |-- dashboard.png
    |-- map.png
    |-- alert_email.png
    |-- eventstream.png
```

Commands

```
mkdir fabric-bike-realtime
cd fabric-bike-realtime
mkdir docs kql dashboards
git init
# add your README.md, kql/queries.kql, screenshots
git add .
git commit -m "Live bike station monitoring on Microsoft Fabric"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/fabric-bike-realtime.git
git push -u origin main
```

Create the empty repo on github.com first (no README), then run these commands. Replace YOUR-USERNAME.

README template

```
# Live Bike-Station Monitoring on Microsoft Fabric

## Problem
Bike-sharing operators need to know instantly which stations are empty or full
so they can move bikes. Daily reports are too slow.

## Architecture
![architecture](docs/architecture.png)

## Tools
Fabric Eventstream, Eventhouse (KQL), Real-Time Dashboard, Activator.

## Data
Built-in Fabric Eventstream sample data "Bicycles" (live, synthetic).

## What I built
1. Eventstream receiving live bike-station events
2. Eventhouse table "bikes" with direct ingestion
3. KQL queries: latest status, empty/full stations, bikes by area, trend
4. Real-Time Dashboard with map and auto refresh
5. Activator email alert when a station has 0 bikes

## Screenshots
(dashboard, map, alert email)

## What I learned
(write 3-4 real points: arg_max for latest row, ingestion_time(), direct
ingestion, alert thresholds)

## Next steps
Add Lakehouse + PySpark bronze/silver/gold, Power BI, own Python producer.
```

For architecture.png: take a screenshot of the diagram in Part 1 of this PDF, or redraw it in draw.io (free).

CHECK: your repo opens on GitHub with README and screenshots.

Troubleshooting

Problem	Fix
Cannot see 'Use sample data' option	Click Add source and look for Sample data in the list (name may be 'Sample data' or 'Sample scenarios').
Data preview is empty	Wait 30 seconds. If still empty, delete the source and add Bicycles sample again.
bikes table has 0 rows	Publish the Eventstream, check the destination is Active, check the table is named bikes, wait 2-3 minutes.
Query says column not found	Run: bikes | getschema. Use the real column names.
Query 2 returns nothing	Maybe no station is empty right now. Try No_Bikes <= 2, or wait a few minutes.
Map tile shows nothing	Make sure the result has Latitude and Longitude columns as numbers (Query 6).
No alert email	Check rule is started, check spam, lower the threshold, wait a few minutes.
Fabric says capacity limit / throttled	Close unused items, wait a few minutes. Delete old test Eventstreams.

Interview Pitch and Resume Lines

30-second explanation

'I built a live streaming project in Microsoft Fabric. Bike-station events come into an Eventstream and are ingested into an Eventhouse. I wrote KQL queries to find the latest status of each station, empty and full stations, and bikes by area. A Real-Time Dashboard with a map refreshes automatically, and Activator sends an email when stations run out of bikes. I used the built-in sample for data, and I can replace it with my own Python producer or a real source.'

Question	Answer
Why Eventhouse for this data?	It is built for fast queries on time-based streaming data and ingests in seconds.
What does arg_max do?	Returns the row with the maximum value of a column per group. I used it to get the latest status per station.
How is Eventstream different from Event Hubs?	Eventstream is Fabric's no-code tool to ingest, transform and route events to many destinations.
How would you improve it?	Add a Lakehouse with bronze/silver/gold tables, Power BI reports, and a real data source.

Resume bullets

- Built a real-time monitoring solution on Microsoft Fabric using Eventstream, Eventhouse (KQL), Real-Time Dashboard and Activator.
- Wrote KQL queries (arg_max, summarize, bin) to track live station availability and surface empty and full stations on a map dashboard.
- Configured automated email alerts with Fabric Activator and documented the project on GitHub.

Checklist and Next Level

Done	Item
[ ]	Workspace bike-rti created
[ ]	Eventhouse bike_eh created
[ ]	Eventstream bike_stream with Bicycles sample data
[ ]	Table bikes receiving data (count keeps growing)
[ ]	KQL queries 1-7 saved
[ ]	Dashboard with map, charts, auto refresh
[ ]	Alert email received
[ ]	GitHub repo pushed with README and screenshots

Level 2 upgrades (do one at a time, only after this works)

- Add a **Lakehouse** destination in the Eventstream and a PySpark notebook for bronze, silver, gold tables.
- Build a **Power BI** report from the gold table.
- Replace the sample with your own **Python producer** (azure-eventhub) sending custom events.
- Move on to the bigger **fraud monitoring** project PDF.

Save trial capacity

- When you finish, take all screenshots first, then stop or delete the Eventstream so it does not keep using capacity.